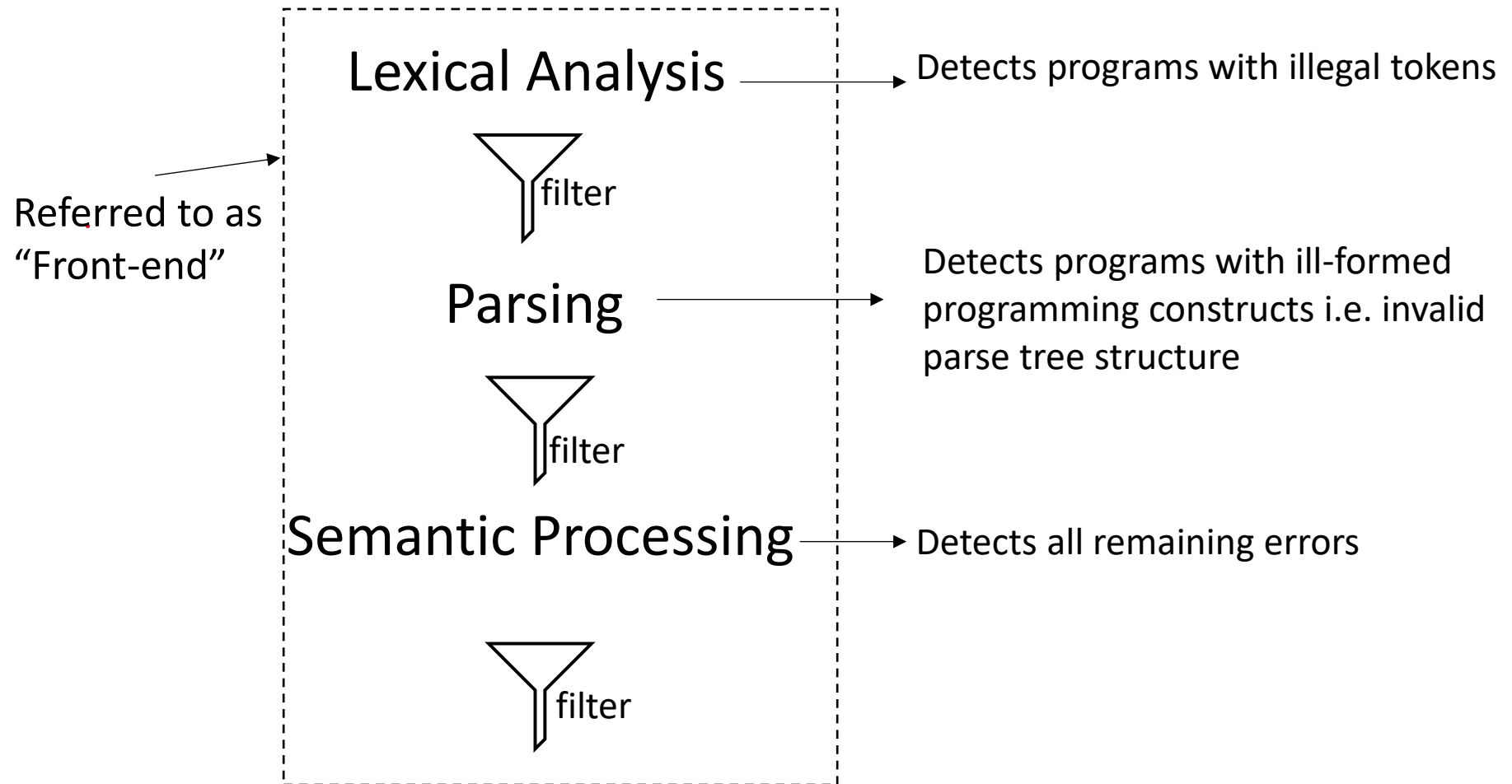


CS620T: Compilers - Principles and Implementation

Autumn 2026

Week 4: Semantic processing

Semantic Processing



Semantic Processing

- Syntax-directed / syntax-driven
 - Routines (called as semantic routines) interpret the meaning of programming constructs based on the syntactic structure
- Routines play a dual role
 - Analysis – Semantic analysis
 - undefined vars, undefined types, uninitialized variables, type errors that can be caught at compile time, unreachable code, etc.
 - Synthesis – Generation of intermediate code
 - 3 address code
- Routines create semantic records to aid the analysis and synthesis

Semantic Processing

- **Syntax-directed translation:** notation for *attaching* program fragments to grammar productions.
 - Program fragments are executed when productions are matched
 - The combined execution of all program fragments produces the translation of the program

e.g. $E \rightarrow E + T$ { print('+') }

Output: program fragments may create AST and 3 Address Codes

- **Attributes:** any 'quality' associated with a terminal and non-terminal e.g. type, number of lines of a code, first line of the code block etc.

Why Semantic Analysis?

- Context-free grammars cannot specify all requirements of a language
 - Identifiers declared before their use (scope)
 - Types in an expression must be consistent

```
STRING str:= "Hello";  
str := str + 2;
```
 - Number of formal and actual parameters of a function must match
 - Reserved keywords cannot be used as identifiers
 - A Class is declared only once in a OO language program, a method of a class can be overridden.
 - ...

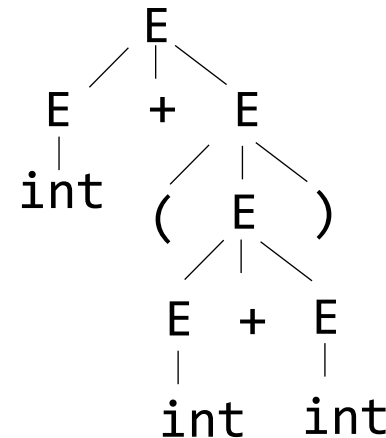
Abstract Syntax Tree

- Abstract Syntax Tree (AST) or Syntax Tree can be the input for semantic analysis.
 - What is Concrete Syntax Tree? – the parse tree
- ASTs are like parse trees but ignore certain details:

E.g. Consider the grammar:

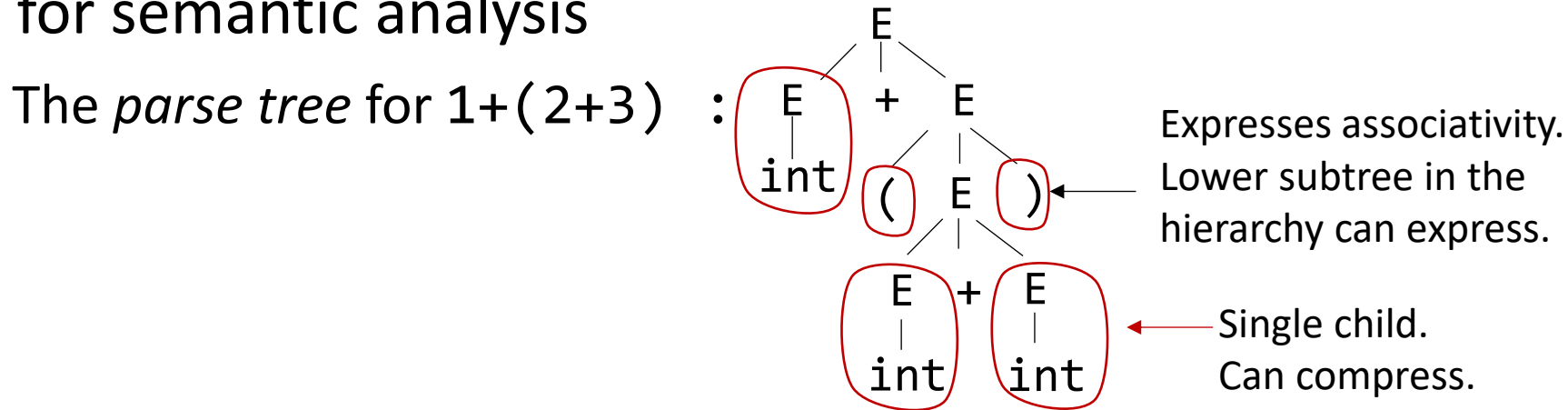
$E \rightarrow E + E$
 $\quad | (E)$
 $\quad | \text{int}$

The parse tree for $1+(2+3)$



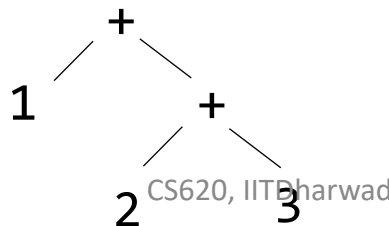
AST - Example

- Not all details (nodes) of the parse tree are helpful for semantic analysis

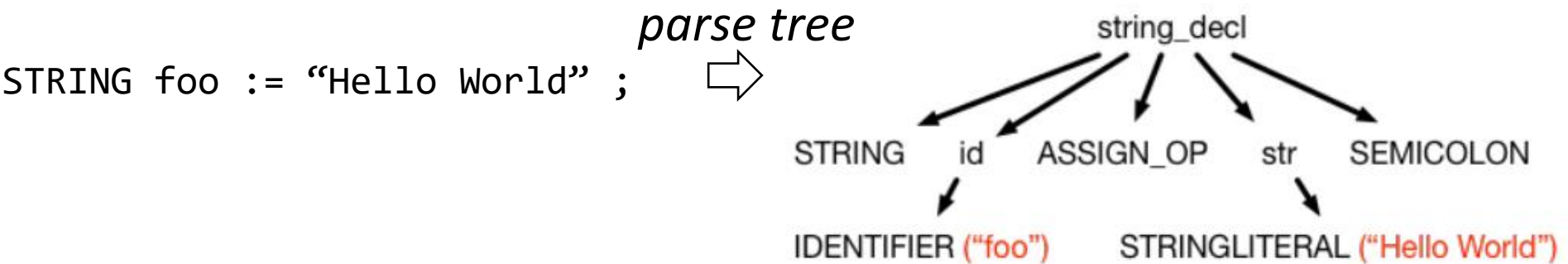


We need to compute the result of the expression. So, a simpler structure is sufficient:

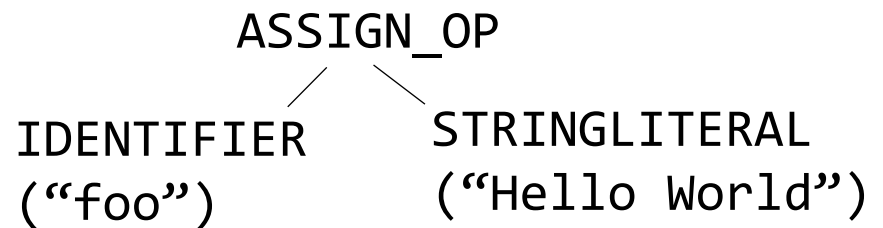
AST for $1+(2+3)$:



AST - Example



$\equiv$ *AST* $\Downarrow$

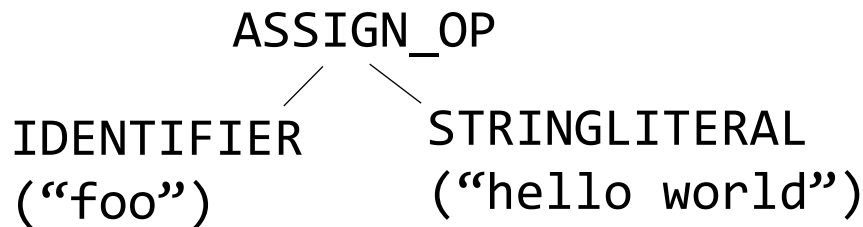


Semantic Analysis – Example

- Context-free grammars cannot specify all requirements of a language
 - Identifiers declared before their use (scope)
 - Types in an expression must be consistent
 - Type checks
 - STRING str:= “Hello”;
 - str := str + 2;
 - Number of formal and actual parameters of a function must match
 - Reserved keywords cannot be used as identifiers
 - A Class is declared only once in a OO language, a method can be overridden.
 - ...

Scope

- **Goal:** matching identifier declarations with uses
- Most languages require this!
- Scope confines the activity of an identifier



What if `foo` is declared as a `STRING` in an enclosing scope but is an `INT` in the current scope?

in different parts of the program:

- Same identifier may refer to different things
- Same identifier may not be accessible

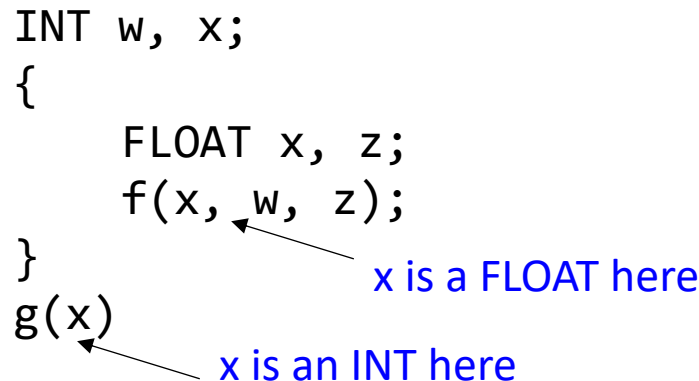
Static Scope

- Most languages are statically scoped
 - Scope depends on only the program text (not runtime behavior)
 - A variable refers to the closest defined instance

```
INT w, x;  
{  
    FLOAT x, z;  
    f(x, w, z);  
}  
g(x)
```

x is a FLOAT here

x is an INT here



Dynamic Scope

- In dynamically scoped languages
 - Scope depends on the execution context
 - A variable refers to the closest enclosing binding in the execution of the program

```
f(){  
    a=4; g();  
}  
g() { print(a); }
```

↑
value of a is 4 here

Exercise: Static vs. Dynamic Scope

```
#define a (x+1) //macro definition
```

```
int x = 2; //global var definition
```

Is x statically scoped or dynamically scoped?

```
//function b definition
```

```
void b() {  
    int x = 1;  
    printf("%d\n", a);  
}
```

```
//function c definition
```

```
void c() {  
    printf("%d\n", a);  
}
```

```
//the main function
```

```
int main() { b(); c(); }
```

Symbol Table

- Data structure that tracks the bindings of identifiers. Specifically, returns the current binding.
 - E.g., stores a mapping of names to types
 - Should provide for efficient retrieval and frequent insertion and deletion of names.
 - Should consider scopes

```
{
    int x = 0;
    //accessing y here should be illegal
    {
        int y = 1;
    }
}
```

- Can use stacks, binary trees, hash maps for implementation

Symbol Table and Classes in OO Language

- Class names may be used before their definition
- Can't use symbol table (to check class definition)
 - Gather all class names first.
 - Check bindings next.
- Semantic analysis is done in multiple passes
- One of the goals of semantic analysis is to create/update data structures that help the next round of analysis



Implies going over the program text multiple times

Semantic Analysis – How?

- Recursive descent of AST
 - Process a node, n
 - Recurse into children of n and process them
 - Finish processing the node, n
- ⇒ Do a postorder processing of the AST
- As you visit a node, you will add information depending upon the analysis performed
 - The information is referred to as attributes of the node

Building AST - Example

- Fully-Parentthesized Expressions (FPE)

- Can build while parsing via bottom-up building of the tree
- Create subtrees, make those subtrees left- and right-children of a newly created root.
- Need to modify the hand-written recursive parser:

if:



token == INTLITERAL, return a reference to newly created node containing a number

else:

store references to nodes that are left- and right- expression subtrees

Create a new node with value = 'OP'

Building AST - Example

This function creates an AST node and adds information that stores the value of an INTLITERAL in the node. A reference to the AST node is returned.

```
TreeNode* IsTerm(Scanner* s, TOKEN tok) {  
    TreeNode* ret = NULL;  
    TOKEN nxtToken = s->GetNextToken();  
    if(nxtToken == tok)  
        ret = CreateTreeNode(nxtToken.val);  
    return ret;  
}
```

Building AST

E1 needs to change because IsTerm returns a TreeNode.*

E1 returns a TreeNode now.*

Recall: E1 is the function that gets called when predicting using the production: $E \rightarrow \text{INTLITERAL}$

```
TreeNode* E1(Scanner* s) {  
  
    return IsTerm(s, INTLITERAL);  
  
}
```

Building AST - Example

- Fully-Parentthesized Expressions (FPE)
 - Can build while parsing via bottom-up building of the tree
 - Create subtrees, make those subtrees left- and right-children of a newly created root.
 - Need to modify the hand-written recursive parser:
 - if:
token == INTLITERAL, return a reference to newly created node containing a number
 - else:
 store references to nodes that are left- and right- expression subtrees
Create a new node with value = 'OP'

Building AST

This function creates an AST node and adds information that stores the value of an op in the node. A reference to the AST node is returned.

Recall: op is the function that gets called when predicting using the production:

op -> ADD | SUB | MUL | DIV

```
TreeNode* OP(Scanner* s, TOKEN tok) {  
    TreeNode* ret = NULL;  
    TOKEN tok = s->GetNextToken();  
    if((tok == ADD) || (tok == SUB) || (tok ==  
MUL) || (tok == DIV))  
        ret = CreateTreeNode(tok.val);  
    return ret;  
}
```

Building AST

This function sets the references to left- and right- expression subtrees if those subtrees are valid FPEs. Returns reference to the AST node corresponding to the op value, NULL otherwise.

Recall: E2 is the function that gets called when predicting using the production: $E \rightarrow (E \text{ op } E)$

```
TreeNode* E2(Scanner* s, TOKEN tok) {  
    TOKEN nxtTok = s->GetNextToken();  
    if(nxtTok == LPAREN) {  
        TreeNode* left = E(s); if(!left) return NULL;  
        TreeNode* root  = OP(s); if(!root) return NULL;  
        TreeNode* right = E(s); if(!right) return NULL;  
        nxtTok = s->GetNextToken();  
        if(nxtTok != RPAREN); return NULL;  
        //set left and right as children of root.  
        return root;  
    }  
}
```

Building AST

E needs to change because E1, E2, and OP return a TreeNode.
E returns a TreeNode* now.*

Recall: E is the higher-level function for a non-terminal that gets called when predicting using either of the productions for E:

$E \rightarrow (E \text{ op } E) \mid \text{INTLITERAL}$

```
TreeNode* E(Scanner* s) {  
  
    TOKEN* prevToken = s->GetCurTokenSequence();  
    TreeNode* ret = E1(s);  
    if(!ret) {  
        s->SetCurTokenSequence(prevToken);  
        ret = E2(s);  
    }  
    return ret;  
}
```

Building AST

```
TreeNode* IsTerm(Scanner* s, TOKEN tok) {
    TreeNode* ret = NULL;
    TOKEN nxtToken = s->GetNextToken();
    if(nxtToken == tok)
        ret = CreateTreeNode(nxtToken.val);
    return ret;
}

TreeNode* E1(Scanner* s) {
    return IsTerm(s, INTLITERAL);
}

TreeNode* E2(Scanner* s) {
    TOKEN nxtTok = s->GetNextToken();
    if(nxtTok == LPAREN) {
        TreeNode* left = E(s);
        if(!left) return NULL;
        TreeNode* root = OP(s);
        if(!root) return NULL;
        TreeNode* right = E(s);
        if(!right) return NULL;
        nxtTok = s->GetNextToken();
        if(nxtTok != RPAREN); return NULL;
        //set left and right as children of root.
        return root;
    }
}
```

Building AST

```
TreeNode* OP(Scanner* s) {
    TreeNode* ret = NULL;
    TOKEN tok = s->GetNextToken();
    if((tok == ADD) || (tok == SUB) || (tok == MUL) || (tok == DIV))
        ret = CreateTreeNode(tok.val);
    return ret;
}

TreeNode* E(Scanner* s) {
    TOKEN* prevToken = s->GetCurTokenSequence();
    TreeNode* ret = E1(s);
    if(!ret) {
        s->SetCurTokenSequence(prevToken);
        ret = E2(s);
    }
    return ret;
}
```

***Start the parser by invoking E().
Value returned is the root of the AST.***

Exercise

- Did we build the AST bottom-up or top-down?

AST Construction with Hand-written Parser

```
TreeNode* E(Scanner* s) {  
    TOKEN* prevToken = s->GetCurTokenSequence();  
    TreeNode* ret = E1(s);  
    if(!ret) {  
        s->SetCurTokenSequence(prevToken);  
        ret = E2(s);  
    }  
    return ret;  
}
```

1. E -> INTLITERAL

2. E -> (E op E)

3. op -> ADD | SUB
| MUL | DIV

Input string: (2+3)

next token



Sequence of tokens given by scanner: LPAREN INTLITERAL ADD INTLITERAL RPAREN

Call stack

E()

Parse tree

E

Start by calling parser function E. Note the call stack contains E(). The parse tree is not constructed. This is a visualization aid.

AST Construction with Hand-written Parser

```
TreeNode* E(Scanner* s) {  
    TOKEN* prevToken = s->GetCurTokenSequence();  
    TreeNode* ret = E1(s);  
    if(!ret) {  
        s->SetCurTokenSequence(prevToken);  
        ret = E2(s);  
    }  
    return ret;  
}
```

1. E -> INTLITERAL

2. E -> (E op E)

3. op -> ADD | SUB
| MUL | DIV

Input string: (2+3)

next token



Sequence of tokens given by scanner: LPAREN INTLITERAL ADD INTLITERAL RPAREN

Call stack

E()
E1()

Parse tree

E
|
INTLITERAL

E() calls E1(). This is like predicting rule 1.

AST Construction with Hand-written Parser

```
TreeNode* E1(Scanner* s) {  
    return IsTerm(s, INTLITERAL);  
}
```

1. $E \rightarrow \text{INTLITERAL}$

2. $E \rightarrow (E \text{ op } E)$

3. $\text{op} \rightarrow \text{ADD} \mid \text{SUB} \mid \text{MUL} \mid \text{DIV}$

Input string: (2+3)

next token



Sequence of tokens given by scanner: LPAREN INTLITERAL ADD INTLITERAL RPAREN

Call stack

E()
E1()
IsTerm()

Parse tree

E
|
INTLITERAL

E1() calls IsTerm() with an expectation that INTLITERAL is the next token.

AST Construction with Hand-written Parser

```
TreeNode* IsTerm(Scanner* s, TOKEN tok) {  
    TreeNode* ret = NULL;  
    TOKEN nxtToken = s->GetNextToken();  
    if(nxtToken == tok)  
        ret = CreateTreeNode(nxtToken.val);  
    return ret;  
}
```

1. $E \rightarrow \text{INTLITERAL}$

2. $E \rightarrow (E \text{ op } E)$

3. $\text{op} \rightarrow \text{ADD} \mid \text{SUB}$

$\mid \text{MUL} \mid \text{DIV}$

Input string: (2+3)

next token



Sequence of tokens given by scanner: LPAREN INTLITERAL ADD INTLITERAL RPAREN

Call stack

Parse tree

IsTerm() calls GetNextToken() which returns LPAREN.

E()
E1()
IsTerm()

E
|
INTLITERAL

AST Construction with Hand-written Parser

```
TreeNode* IsTerm(Scanner* s, TOKEN tok) {  
    TreeNode* ret = NULL;  
    TOKEN nxtToken = s->GetNextToken();  
    if(nxtToken == tok)  
        ret = CreateTreeNode(nxtToken.val);  
    return ret;  
}
```

1.E -> INTLITERAL

2.E -> (E op E)

3.op -> ADD | SUB

| MUL | DIV

Input string: (2+3)

next token



Sequence of tokens given by scanner: LPAREN INTLITERAL ADD INTLITERAL RPAREN

Call stack

E()
E1()
IsTerm()

Parse tree

E
|
INTLITERAL

IsTerm() calls GetNextToken() which returns LPAREN.
In addition, GetNextToken() advances the 'next token' pointer.

AST Construction with Hand-written Parser

```
TreeNode* IsTerm(Scanner* s, TOKEN tok) {  
    TreeNode* ret = NULL;  
    TOKEN nxtToken = s->GetNextToken();  
    if(nxtToken == tok)  
        ret = CreateTreeNode(nxtToken.val);  
    return ret;  
}
```

1. $E \rightarrow \text{INTLITERAL}$

2. $E \rightarrow (E \text{ op } E)$

3. $\text{op} \rightarrow \text{ADD} \mid \text{SUB} \mid \text{MUL} \mid \text{DIV}$

Input string: (2+3)

next token



Sequence of tokens given by scanner: LPAREN INTLITERAL ADD INTLITERAL RPAREN

Call stack

E()
E1()
IsTerm()

Parse tree

E
|
INTLITERAL

IsTerm() calls GetNextToken() which returns LPAREN. In addition, GetNextToken() advances the 'next token' pointer. There is a mismatch (IsTerm expects INTLITERAL (tok=INTLITERAL) but nextToken is LPAREN. So returns NULL.

AST Construction with Hand-written Parser

```
TreeNode* E1(Scanner* s) {  
    return IsTerm(s, INTLITERAL);  
}
```

1. $E \rightarrow \text{INTLITERAL}$

2. $E \rightarrow (E \text{ op } E)$

3. $\text{op} \rightarrow \text{ADD} \mid \text{SUB}$
 $\mid \text{MUL} \mid \text{DIV}$

Input string: (2+3)

next token

Sequence of tokens given by scanner: LPAREN INTLITERAL ADD INTLITERAL RPAREN

Call stack

E()
E1()

Parse tree

E
|
INTLITERAL

E1 returns NULL because IsTerm returned NULL
(note that an entry from call stack is popped off)

AST Construction with Hand-written Parser

```
TreeNode* E(Scanner* s) {  
    TOKEN* prevToken = s->GetCurTokenSequence();  
    TreeNode* ret = E1(s);  
    if(!ret) {  
        s->SetCurTokenSequence(prevToken);  
        ret = E2(s);  
    }  
    return ret;  
}
```

1. E -> INTLITERAL

2. E -> (E op E)

3. op -> ADD | SUB

| MUL | DIV

Input string: (2+3)

next token



Sequence of tokens given by scanner: LPAREN INTLITERAL ADD INTLITERAL RPAREN

Call stack

E()

Parse tree

E

E1 returning NULL implies that predicting rule 1 failed. ret is NULL (note that an entry from call stack is popped off).

AST Construction with Hand-written Parser

```
TreeNode* E(Scanner* s) {  
    TOKEN* prevToken = s->GetCurTokenSequence();  
    TreeNode* ret = E1(s);  
    if(!ret) {  
        s->SetCurTokenSequence(prevToken);  
        ret = E2(s);  
    }  
    return ret;  
}
```

1. E -> INTLITERAL

2. E -> (E op E)

3. op -> ADD | SUB
| MUL | DIV

Input string: (2+3)

next token



Sequence of tokens given by scanner: LPAREN INTLITERAL ADD INTLITERAL RPAREN

Call stack

E()

Parse tree

E

E restores 'next token' pointer back to the saved pointer prevToken (using SetCurTokenSequence())

AST Construction with Hand-written Parser

```
TreeNode* E(Scanner* s) {  
    TOKEN* prevToken = s->GetCurTokenSequence();  
    TreeNode* ret = E1(s);  
    if(!ret) {  
        s->SetCurTokenSequence(prevToken);  
        ret = E2(s);  
    }  
    return ret;  
}
```

1. $E \rightarrow \text{INTLITERAL}$

2. $E \rightarrow (E \text{ op } E)$

3. $\text{op} \rightarrow \text{ADD} \mid \text{SUB}$

$\mid \text{MUL} \mid \text{DIV}$

Input string: (2+3)

next token

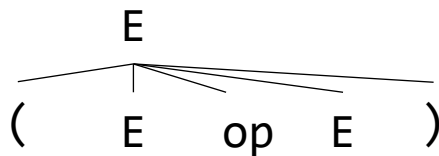


Sequence of tokens given by scanner: LPAREN INTLITERAL ADD INTLITERAL RPAREN

Call stack

E()
E2()

Parse tree



Calls E2. This is like predicting Rule 2. Note the parse tree. Again, the tree is not constructed and is used only to visualize the parsing

AST Construction with Hand-written Parser

```
TreeNode* E2(Scanner* s) {  
    TOKEN nxtTok = s->GetNextToken();  
    if(nxtTok == LPAREN) {  
        TreeNode* left = E(s);  
        if(!left) return NULL;  
        TreeNode* root = OP(s);  
        ...  
    }
```

1. E -> INTLITERAL

2. E -> (E op E)

3. op -> ADD | SUB
| MUL | DIV

Input string: (2+3)

next token

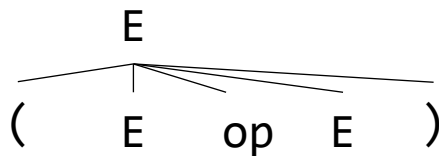


Sequence of tokens given by scanner: LPAREN INTLITERAL ADD INTLITERAL RPAREN

Call stack

E()
E2()

Parse tree



E2 check for LPAREN succeeds (note 'next token' pointer is moved forward after the call to GetNextToken().)

AST Construction with Hand-written Parser

```
TreeNode* E2(Scanner* s) {  
    TOKEN nxtTok = s->GetNextToken();  
    if(nxtTok == LPAREN) {  
        TreeNode* left = E(s);  
        if(!left) return NULL;  
        TreeNode* root = OP(s);  
        ...  
    }
```

1. E -> INTLITERAL

2. E -> (E op E)

3. op -> ADD | SUB
| MUL | DIV

Input string: (2+3)

next token

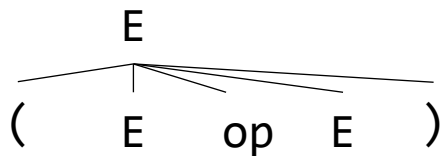


Sequence of tokens given by scanner: LPAREN INTLITERAL ADD INTLITERAL RPAREN

Call stack

E()
E2()

Parse tree



Calls E()

AST Construction with Hand-written Parser

```
TreeNode* E(Scanner* s) {  
    TOKEN* prevToken = s->GetCurTokenSequence();  
    TreeNode* ret = E1(s);  
    if(!ret) {  
        s->SetCurTokenSequence(prevToken);  
        ret = E2(s);  
    }  
    return ret;  
}
```

1. **E -> INTLITERAL**

2. **E -> (E op E)**

3. **op -> ADD | SUB
| MUL | DIV**

Input string: (2+3)

next token

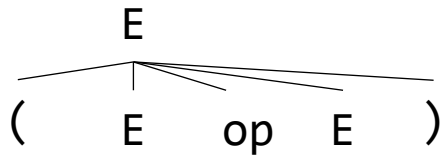


Sequence of tokens given by scanner: **LPAREN INTLITERAL ADD INTLITERAL RPAREN**

Call stack

E()
E2()
E()

Parse tree



E calls E1() to predict rule 1 to match the E following (in the parse tree

AST Construction with Hand-written Parser

```
TreeNode* E1(Scanner* s) {  
    return IsTerm(s, INTLITERAL);  
}
```

1. $E \rightarrow \text{INTLITERAL}$

2. $E \rightarrow (E \text{ op } E)$

3. $\text{op} \rightarrow \text{ADD} \mid \text{SUB} \mid \text{MUL} \mid \text{DIV}$

Input string: (2+3)

next token

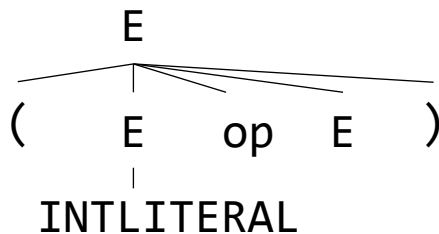


Sequence of tokens given by scanner: LPAREN INTLITERAL ADD INTLITERAL RPAREN

Call stack

E()
E2()
E()
E1()

Parse tree



E1 calls IsTerm() and expects INTLITERAL

AST Construction with Hand-written Parser

```
TreeNode* IsTerm(Scanner* s, TOKEN tok) {  
    TreeNode* ret = NULL;  
    TOKEN nxtToken = s->GetNextToken();  
    if(nxtToken == tok)  
        ret = CreateTreeNode(nxtToken.val);  
    return ret;  
}
```

1. $E \rightarrow \text{INTLITERAL}$

2. $E \rightarrow (E \text{ op } E)$

3. $\text{op} \rightarrow \text{ADD} \mid \text{SUB} \mid \text{MUL} \mid \text{DIV}$

Input string: (2+3)

Sequence of tokens given by scanner: LPAREN INTLITERAL ADD INTLITERAL RPAREN

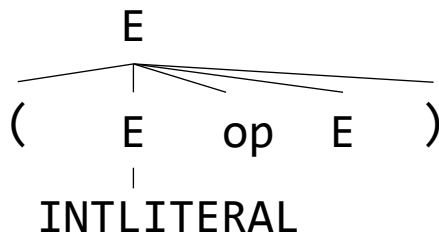
next token



Call stack

E()
E2()
E()
E1()
IsTerm()

Parse tree



Call to GetNextToken() in IsTerm() now returns INTLITERAL and advances the pointer. The if condition is true.

AST Construction with Hand-written Parser

```
TreeNode* IsTerm(Scanner* s, TOKEN tok) {  
    TreeNode* ret = NULL;  
    TOKEN nxtToken = s->GetNextToken();  
    if(nxtToken == tok)  
        ret = CreateTreeNode(nxtToken.val);  
    return ret;  
}
```

1. $E \rightarrow \text{INTLITERAL}$

2. $E \rightarrow (E \text{ op } E)$

3. $\text{op} \rightarrow \text{ADD} \mid \text{SUB}$

$\mid \text{MUL} \mid \text{DIV}$

Input string: (2+3)

next token

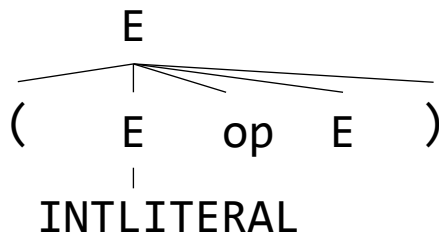


Sequence of tokens given by scanner: LPAREN INTLITERAL ADD INTLITERAL RPAREN

Call stack

E()
E2()
E()
E1()
IsTerm()

Parse tree



AST Node is created and stores the INTLITERAL's value returned by the scanner (via s->GetNextToken()).

Note that in this example we are storing the string corresponding to the integer val.

AST Construction with Hand-written Parser

```
TreeNode* IsTerm(Scanner* s, TOKEN tok) {  
    TreeNode* ret = NULL;  
    TOKEN nxtToken = s->GetNextToken();  
    if(nxtToken == tok)  
        ret = CreateTreeNode(nxtToken.val);  
    return ret;  
}
```

1. $E \rightarrow \text{INTLITERAL}$
2. $E \rightarrow (E \text{ op } E)$
3. $\text{op} \rightarrow \text{ADD} \mid \text{SUB} \mid \text{MUL} \mid \text{DIV}$

Input string: (2+3)

next token

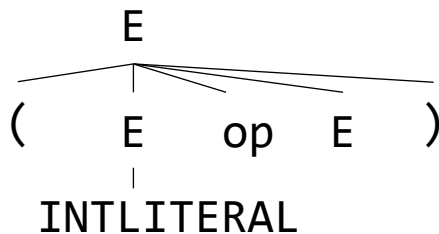


Sequence of tokens given by scanner: LPAREN INTLITERAL ADD INTLITERAL RPAREN

Call stack

E()
E2()
E()
E1()
IsTerm()

Parse tree



IsTerm() returns the pointer to the tree node created.

AST Construction with Hand-written Parser

```
TreeNode* E1(Scanner* s) {  
    return IsTerm(s, INTLITERAL);  
}
```

1. $E \rightarrow \text{INTLITERAL}$

2. $E \rightarrow (E \text{ op } E)$

3. $\text{op} \rightarrow \text{ADD} \mid \text{SUB}$

$\mid \text{MUL} \mid \text{DIV}$

Input string: (2+3)

next token



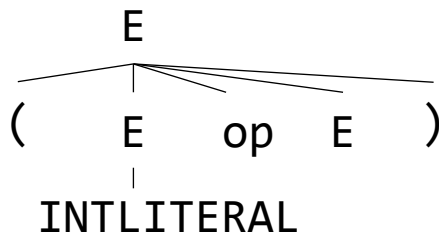
Sequence of tokens given by scanner: LPAREN INTLITERAL ADD INTLITERAL RPAREN

E1 returns the pointer to the tree node.

Call stack

E()
E2()
E()
E1()

Parse tree



AST Construction with Hand-written Parser

```
TreeNode* E(Scanner* s) {  
    TOKEN* prevToken = s->GetCurTokenSequence();  
    TreeNode* ret = E1(s);  
    if(!ret) {  
        s->SetCurTokenSequence(prevToken);  
        ret = E2(s);  
    }  
    return ret;  
}
```

1. E -> INTLITERAL

2. E -> (E op E)

3. op -> ADD | SUB
| MUL | DIV

Input string: (2+3)

next token



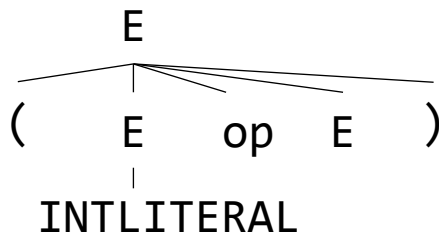
Sequence of tokens given by scanner: LPAREN INTLITERAL ADD INTLITERAL RPAREN

E returns the pointer to the tree node.

Call stack

E()
E2()
E()

Parse tree



AST Construction with Hand-written Parser

```
TreeNode* E2(Scanner* s) {  
    TOKEN nxtTok = s->GetNextToken();  
    if(nxtTok == LPAREN) {  
        TreeNode* left = E(s);  
        if(!left) return NULL;  
        TreeNode* root = OP(s);  
        ...  
    }
```

1. E -> INTLITERAL

2. E -> (E op E)

3. op -> ADD | SUB
| MUL | DIV

Input string: (2+3)

next token

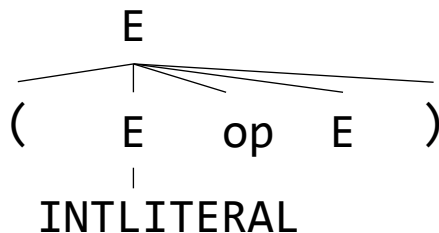


Sequence of tokens given by scanner: LPAREN INTLITERAL ADD INTLITERAL RPAREN

Call stack

E()
E2()

Parse tree



E2() now has a non-null value set for left (left is a pointer to the root of the left subtree). The if condition is false.

AST Construction with Hand-written Parser

```
TreeNode* E2(Scanner* s) {  
    TOKEN nxtTok = s->GetNextToken();  
    if(nxtTok == LPAREN) {  
        TreeNode* left = E(s);  
        if(!left) return NULL;  
        TreeNode* root = OP(s);  
        ...  
    }
```

1. E -> INTLITERAL

2. E -> (E op E)

3. op -> ADD | SUB
| MUL | DIV

Input string: (2+3)

next token



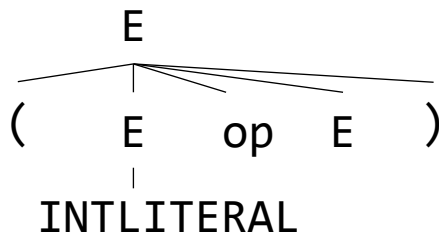
Sequence of tokens given by scanner: LPAREN INTLITERAL ADD INTLITERAL RPAREN

E2() calls Op()

Call stack

E()
E2()

Parse tree



AST Construction with Hand-written Parser

```
TreeNode* OP(Scanner* s) {  
    TreeNode* ret = NULL;  
    TOKEN tok = s->GetNextToken();  
    if((tok == ADD) || (tok == SUB) || (tok == MUL) ||  
        (tok == DIV))  
        ret = CreateTreeNode(tok.val);  
    return ret;  
}
```

1. E → INTLITERAL

2. E → (E op E)

3. op → ADD | SUB
| MUL | DIV

Input string: (2+3)

next token

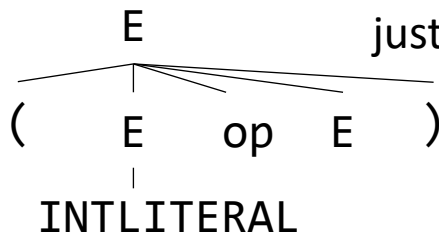


Sequence of tokens given by scanner: LPAREN INTLITERAL ADD INTLITERAL RPAREN

Call stack

E()
E2()
Op()

Parse tree



Op() first matches the next token with ADD and creates a node with value '+'. It then returns a pointer to the tree node just created. (note the next token pointer is also advanced)



AST Construction with Hand-written Parser

```
E2(){  
    ...  
    TreeNode* root = OP(s);  
    if(!root) return NULL;  
    TreeNode* right = E(s)  
    if(!right) return NULL;  
    nxtTok = s->GetNextToken();  
    if(nxtTok != RPAREN); return NULL;  
    //set left and right as children of root.  
    return root; }
```

1.E -> INTLITERAL

2.E -> (E op E)

3.op -> ADD | SUB

| MUL | DIV

Input string: (2+3)

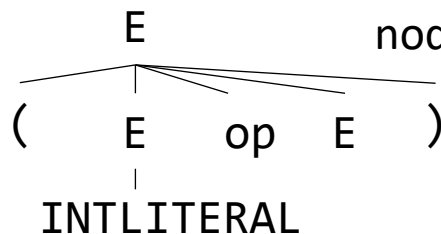
next token

Sequence of tokens given by scanner: **LPAREN INTLITERAL ADD INTLITERAL RPAREN**

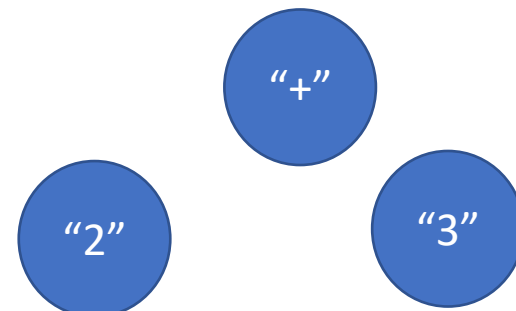
Call stack

E()
E2()

Parse tree



Now 'root' in E2 is set to a non-null value. E() is called next. E() in turn calls E1(), which calls IsTerm() that creates a tree node with value "3" and returns a pointer to it.



AST Construction with Hand-written Parser

```
E2(){  
    ...  
    TreeNode* root = OP(s);  
    if(!root) return NULL;  
    TreeNode* right = E(s);  
    if(!right) return NULL;  
    nextTok = s->GetNextToken();  
    if(nextTok != RPAREN); return NULL;  
    //set left and right as children of root.  
    return root; }
```

1.E -> INTLITERAL

2.E -> (E op E)

3.op -> ADD | SUB

| MUL | DIV

Input string: (2+3)

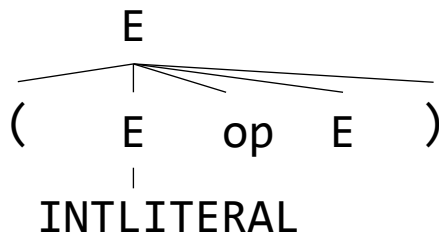
Sequence of tokens given by scanner: LPAREN INTLITERAL ADD INTLITERAL RPAREN

next token
↓

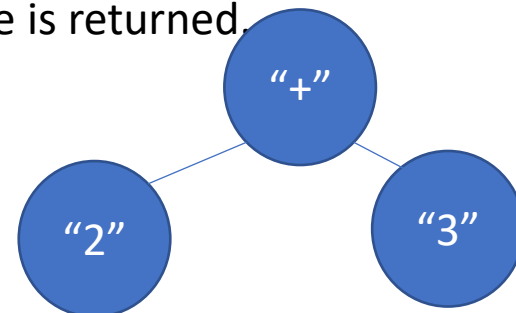
Call stack

E()
E2()

Parse tree



Lastly, the call to GetNextToken() in E2() returns RPAREN and the following if condition fails. Following this failure, the left and right child pointers of the 'root' node are set and the root node is returned.



Observations - AST Construction with Hand-written Parser

1. The AST is created bottom-up
2. Value associated with INTLITERAL/OP is added as information to the AST node
3. Pointer/reference to AST node is returned / passed up the parse tree

Identifying Semantic Actions for FPE Grammar

- What did we do when we saw an INTLITERAL?
 - Create a TreeNode
 - Initialize it with a value (string equivalent of INTLITERAL in this case)
 - Return a pointer to TreeNode

```

E -> INTLITERAL      triggers
    ──────────────────>
        TreeNode* E1(Scanner* s) {
            return IsTerm(s, INTLITERAL);
        }

        ↓

        TreeNode* IsTerm(Scanner* s, TOKEN tok) {
            TreeNode* ret = NULL;
            TOKEN nxtToken = s->GetNextToken();
            if(nxtToken == tok)
                ret = CreateTreeNode(nxtToken.val);
            return ret;
        }

```

Identifying Semantic Actions for FPE Grammar

- What did we do when we saw an E (parenthesized expression)?
 - Create an AST node with two children. The node contains the binary operator OP stored as a string. Children point to roots of subtrees representing E.

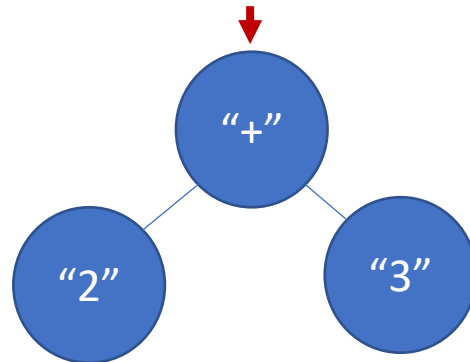
$E \rightarrow (E \text{ op } E)$ $\xrightarrow{\text{triggers}}$

```
TreeNode* E2(Scanner* s) {  
    TOKEN nxtTok = s->GetNextToken();  
    if(nxtTok == LPAREN) {  
        TreeNode* left = E(s);  
        if(!left) return NULL;  
        TreeNode* root = OP(s);  
        if(!root) return NULL;  
        TreeNode* right = E(s);  
        if(!right) return NULL;  
        nxtTok = s->GetNextToken();  
        if(nxtTok != RPAREN); return NULL;  
        //set left and right as children of root.  
        return root;  
    }  
}
```

CS620, MIT Dhanwad

Identifying Semantic Actions for FPE Grammar

- What did we do when we saw an E (parenthesized expression)?
 - Create an AST node with two children. The node contains the binary operator OP stored as a string. Children point to roots of subtrees representing E.
 - Returned reference to root



Identifying Semantic Actions for FPE Grammar

- We can capture the semantic actions identified in the previous slides for INTLITERAL and parenthesized E with the help of notations augmenting grammar rules

Syntax Directed Definition

- Notation containing CFG augmented with attributes and rules

- E.g.

$E \rightarrow \text{INTLITERAL}$	$E.\text{val} = \text{INTLITERAL}.\text{val}$
$E \rightarrow (E \text{ op } E)$	$E.\text{val} = E_1.\text{val} \text{ op } E_2.\text{val}$
$\text{op} \rightarrow \text{ADD}$	$\text{op}.\text{val} = \text{ADD}.\text{val}$
SUB	$\text{op}.\text{val} = \text{SUB}.\text{val}$
MUL	$\text{op}.\text{val} = \text{MUL}.\text{val}$
DIV	$\text{op}.\text{val} = \text{DIV}.\text{val}$

Syntax Directed Definition

- Being more precise (w.r.t. our example)
- E.g.

<code>E -> INTLITERAL</code>	<code>E.node = new TreeNode(INTLITERAL.val)</code>
<code>E -> (E op E)</code>	<code>E.node = TreeNode(op.node, E₁.node, E₂.node)</code>
<code>op -> ADD</code>	<code>op.node = new TreeNode("+")</code>
<code> SUB</code>	<code>op.node = new TreeNode("-");</code>
<code> MUL</code>	<code>op.node = new TreeNode("*");</code>
<code> DIV</code>	<code>op.node = new TreeNode("/");</code>

- Attributes are of two types: Synthesized, Inherited

Syntax Directed Translation

- Complementary notation to SDDs containing CFG augmented with program fragments
- E.g.

E -> INTLITERAL	{E.yylval = INTLITERAL.yylval;}
E -> (E op E)	{E.yylval = eval_binary(E ₁ .yylval, op, E ₂ .yylval)}
op -> ADD	{op.yylval = ADD.yylval}
SUB	{op.yylval = SUB.yylval}
MUL	{op.yylval = MUL.yylval }
DIV	{op.yylval = DIV.yylval}
- Less readable than SDD. However, more efficient for optimizing